

Refatoramento de Código

Exemplo extraído do livro *Refactoring: Improving the Design of Existing Code* (Fowler, 1999)
Remodelado para a disciplina de Padrões de Projeto de Software

Parte 1: Código Original

I. Objetivos:

- Aprender algumas das regras básicas nas quais o programador deve se apoiar ao projetar software;
- Adotar princípios de um bom projeto OO
- Acompanhar um exemplo de refatoramento de software, transformando um software de qualidade pobre em um de melhor qualidade.

II. Regras básicas de um design de software

Criar um bom design de software é algo que se adquire com a experiência. Pode ser considerada como um dos **exercícios mais difíceis** da programação. Basicamente, consiste em **criar abstrações** para:

- a) Determinar quais classes devem ser criadas
- b) Estabelecer quais responsabilidades (métodos) devem ser assumidas por cada classe;
- c) Determinar quais os relacionamentos uma classe deve estabelecer com outras.

III. Princípios GRASP

Neste exercício, exploraremos quatro conceitos por trás dos princípios GRASP:

- **Expert Information:** **colocar as responsabilidades com os dados**, ou seja, atribuir uma responsabilidade ao expert de informação, classe que possui a informação necessária para preencher a responsabilidade.
- **Fraco Acoplamento:** medida para mensurar o quão fortemente uma classe possui ou depende de outra classe. O desejado é um cenário de **fraco acoplamento**.
- **Alta Coesão:** medida empregada para interpretar o qual relacionadas ou focadas estão as responsabilidades de uma classe. O desejado é um cenário de **alta coesão**. Uma classe com **baixa coesão** assume responsabilidades que pertencem a outras classes, logo deveriam ser delegadas a outras classes.
- **Creator:** determina qual classe deve ser responsável por criar outros objetos

IV. Conhecendo o código inicial do Sistema de Locadora

O problema abordado neste exercício é um sistema simples que calcula e emite um extrato de um cliente em uma locadora de veículos, no qual os clientes pagam um valor de diária para cada dia alugado. O diagrama UML das classes principais está disposto na Figura 1.

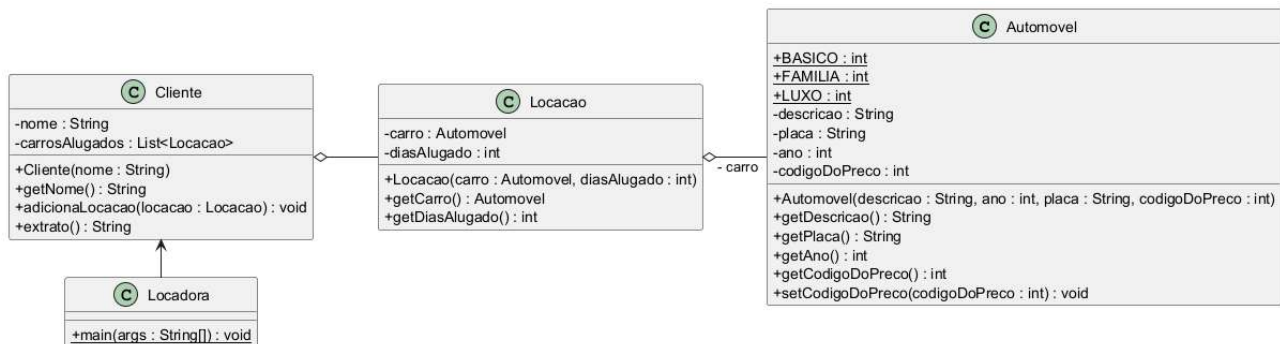


Figura 1. Diagrama de Classes da Locadora de Automóveis.

A classe **Automovel** é usada apenas para encapsular os atributos. O código é ilustrado a seguir

```
public class Automovel {
    public static final int BASICO = 0;
    public static final int FAMILIA = 1;
    public static final int LUXO = 2;

    private String descricao;
    private String placa;
    private int ano; // Ano de fabricação do automóvel
    private int codigoDoPreco;

    public Automovel(String descricao, String placa, int ano,
                     int codigoDoPreco) {
        this.descricao = descricao;
        this.placa = placa;
        this.ano = ano;
        this.codigoDoPreco = codigoDoPreco;
    }

    public String getDescricao() {
        return descricao;
    }

    public String getPlaca() {
        return placa;
    }

    public int getAno() {
        return ano;
    }
}
```

```

    public int getCodigoDoPreco() {
        return codigoDoPreco;
    }

    public void setCodigoDoPreco(int codigoDoPreco) {
        this.codigoDoPreco = codigoDoPreco;
    }
}

```

A classe **Locacao** representa a informação de um aluguel de um automóvel por um certo número de dias.

```

public class Locacao {
    private Automovel carro;
    private int diasAlugado;

    public Locacao(Automovel carro, int diasAlugado) {
        this.carro = carro;
        this.diasAlugado = diasAlugado;
    }

    public Automovel getCarro() {
        return carro;
    }

    public int getDiasAlugado() {
        return diasAlugado;
    }
}

```

A classe **Cliente** representa um cliente da locadora. Internamente, o cliente emite o extrato de seus aluguéis de acordo com o preço de locação. Também determina uma lógica para cálculo de pontos de fidelidade. Os pontos de fidelidade servirão para atribuir algum tipo de desconto ao cliente no futuro.

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Cliente {
    private String nome;
    private List<Locacao> carrosAlugados = new ArrayList<Locacao>();

    public Cliente(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}

```

```

public void adicionaLocacao(Locacao locacao) {
    carrosAlugados.add(locacao);
}

public String extrato() {
    final String fimDeLinha = System.getProperty("line.separator");
    double valorTotal = 0.0;
    int pontosDeAlugadorFrequente = 0;
    int sequencia = 0;

    Iterator<Locacao> locacoes = carrosAlugados.iterator();
    String resultado = "Registro de Aluguéis de " + getNome() +
fimDeLinha;
    resultado += String.format("Seq Automovel          Ano da
Locacao Valor Pago"+fimDeLinha);
    resultado += String.format("=== =====
===== ====="+fimDeLinha);

    while(locacoes.hasNext()) {
        double valorCorrente = 0.0;
        Locacao cada = locacoes.next();

        // determina valores para cada linha
        switch(cada.getCarro().getCodigoDoPreco()) {
            case Automovel.BASICO: // R$ 90.00 por dia
                valorCorrente += cada.getDiasAlugado() * 90.00;
                break;
            case Automovel.FAMILIA: // R$ 130.00 por dia
                valorCorrente += cada.getDiasAlugado() * 130.00;
                break;
            case Automovel.LUXO: // R$ 200.00 por dia.
                valorCorrente += cada.getDiasAlugado() * 200.00;
                // Acima de 4 diárias tem 10% de desconto
                if(cada.getDiasAlugado() > 4) {
                    valorCorrente *= 0.9
                }
                break;
        } //switch

        // trata de pontos de alugador frequente
        pontosDeAlugadorFrequente++;
        // adiciona bonus para aluguel de um lançamento por pelo
menos 2 dias
        if(cada.getCarro().getCodigoDoPreco() == Automovel.LUXO &&
cada.getDiasAlugado() > 2) {
            pontosDeAlugadorFrequente+=2;
        }

        // mostra valores para este aluguel
        sequencia++;
        resultado += String.format "%02d. %-20s          %4d          R$
%8.2f"+fimDeLinha,sequencia, cada.getCarro().getDescricao(),
cada.getCarro().getAno(), valorCorrente );
        valorTotal += valorCorrente;
    } // while
    // adiciona rodapé

```

```

        resultado += "===== " +
fimDeLinha;
        resultado += String.format "Valor Acumulado em
diárias.....: R$ %8.2f" + fimDeLinha, valorTotal);
        resultado += "Voce acumulou " + pontosDeAlugadorFrequente +
        " pontos de alugador frequente";
        return resultado;
    }
}

```

Por fim, a classe **Locadora** é responsável por colocar o programa para funcionar.

```

public class Locadora {
    public static void main(String[] args) {
        Cliente c1 = new Cliente("Alex Sandro");

        c1.adicionalocacao(new Locacao(new Automovel("Toyota Corolla xEi",
2021, "PLN0525", Automovel.LUXO), 10));
        c1.adicionalocacao(new Locacao(new Automovel("Fiat Mobi", 2021,
"JPA2464", Automovel.BASICO), 2));
        c1.adicionalocacao(new Locacao(new Automovel("BMW Série 7", 2022,
"UBA0808", Automovel.LUXO), 30));
        c1.adicionalocacao(new Locacao(new Automovel("Fiat Siena", 2023,
"ABC0001", Automovel.FAMILIA), 4));
        c1.adicionalocacao(new Locacao(new Automovel("Honda HRV", 2024,
"KJD9745", Automovel.FAMILIA), 10));
        c1.adicionalocacao(new Locacao(new Automovel("Volkswagen Gol", 2024,
"JJJ0055", Automovel.BASICO), 3));

        System.out.println(c1.extrato());
    }
}

```

A saída do programa será:

```

Registro de Locações de Alex Sandro
Seq Automovel      Ano  Diarias  Valor Pago
=== =====
01. Toyota Corolla xEi  2021   10    R$  1800,00
02. Fiat Mobi        2021    2    R$   180,00
03. BMW Série 7      2022   30    R$  5400,00
04. Fiat Siena       2023    4    R$   520,00
05. Honda HRV        2024   10    R$  1300,00
06. Volkswagen Gol   2024    3    R$   270,00
=====
Valor Acumulado em diárias.....: R$  9470,00
Voce acumulou 10 pontos de locador frequente

```

V. Tarefa

De acordo com o código apresentado, identifique as fragilidades que podem ser vistas como más práticas de programação ou violação de princípios de design. Além disso, reflita sobre as seguintes questões:

- (a) O sistema está pronto quando uma nova classificação de carro for adicionada (e.g. **Premium**)?
- (b) E quando houver reajuste de preço? O código está pronto para executar facilmente um novo esquema de pontos de alugador frequente?